

REPORTE DE PRÁCTICA: MODULARIDAD, MANEJO DE ARREGLOS Y PERSISTENCIA EN ARCHIVOS DE TEXTO

Asignatura:	Programación Estructurada
Estudiante:	Aldo Ezequiel Rodriguez Godoy
Código:	224097617
Sección:	D04
Profesor:	JORGE ERNESTO LOPEZ ARCE DELGADO
Fecha:	17 de abril del 2026

INTRODUCCIÓN

En el diseño de sistemas de software bajo el paradigma estructurado, la descomposición modular representa una de las estrategias más eficaces para abatir la complejidad algorítmica. Este enfoque permite fragmentar un problema complejo en subprogramas independientes o funciones, facilitando la legibilidad, la reutilización de código y el aislamiento de errores. Cuando estas funciones procesan colecciones de datos continuas, como los arreglos numéricos unidimensionales, el lenguaje C opera bajo una semántica de transferencia por referencia implícita. Esto significa que al pasar un arreglo como argumento a una función, se transmite la dirección de memoria de su primer elemento, permitiendo modificaciones directas y optimizando el uso de la memoria RAM al evitar la duplicación de estructuras completas en la pila.

Por otra parte, la volatilidad de los datos en la memoria del sistema exige mecanismos robustos de persistencia. El lenguaje C soluciona este requerimiento mediante flujos de archivos gestionados a través del apuntador estructurado de tipo `FILE *`. La correcta apertura de canales mediante `fopen` bajo modos de escritura restrictivos (como el modo `"w"`), la transferencia formateada con buffers de texto empleando `fprintf`, y el cierre obligatorio de canales con `fclose` constituyen el flujo estándar de entrada/salida a nivel de archivos de texto plano. El análisis preciso del formateo de salida es crucial; una omisión en los delimitadores visuales (como espacios o saltos de línea) durante la serialización puede fundir múltiples datos lógicos en una sola cadena ambigua, comprometiendo la integridad y legibilidad de la información almacenada.

OBJETIVO

Objetivo General

Diseñar e implementar un programa modular en lenguaje C que capture y procese un conjunto lineal de datos enteros, aislando elementos específicos mediante un algoritmo de filtrado numérico para posteriormente serializar y resguardar la información resultante en un archivo de texto plano externo.

Objetivos Particulares

- Descomponer la solución algorítmica en funciones especializadas con tareas únicas (``capturar``, ``mostrar``, ``filtrar`` y ``guardar``) para consolidar buenas prácticas de modularidad.

- Aplicar el paso de arreglos por referencia hacia subprogramas para realizar modificaciones lógicas en tiempo real sobre colecciones numéricas unidimensionales.
- Implementar flujos de control seguros para la creación y apertura de archivos mediante el puntero `FILE *`, asegurando la separación correcta de datos formateados con espacios divisorios y controlando fallos potenciales de apertura del sistema operativo.

DESARROLLO

El desarrollo metodológico de la práctica consistió en la codificación, optimización y estructuración modular del software. Para cumplir con los requerimientos técnicos y corregir deficiencias de serialización física, el algoritmo fue distribuido a través de componentes funcionales específicos dentro del archivo fuente principal:

1. Inicialización y Abstracción de Datos (`#define`)

Se implementó la directiva de preprocesamiento `#define TAMANO 6` para parametrizar de forma centralizada la longitud de los arreglos numéricos en el programa. Esta práctica de ingeniería de software previene el uso de "números mágicos" hardcoded en el flujo secuencial, facilitando el escalamiento futuro del tamaño del vector desde un único punto crítico de control.

2. Modularización del Ciclo de Captura y Despliegue

La función `capturar` recorre de forma interactiva las casillas del arreglo original mediante un bucle `for`, solicitando al usuario cada valor numérico mediante `scanf`. Paralelamente, la función `mostrar` recibe el apuntador base del arreglo y su dimensión lógica actual para imprimir secuencialmente en la terminal de comandos el contenido del vector envuelto en un salto de línea terminal.

3. Diseño del Algoritmo de Filtrado

La función `filtrar` implementa la lógica matemática de discriminación de elementos. Utilizando el operador de residuo aritmético (`origen[i] % 2 != 0`), evalúa cada posición del arreglo original para identificar números impares. Aquellos elementos que cumplen la condición lógica son copiados de forma contigua en el arreglo de `destino`, manejando un índice de control independiente `j` que avanza de manera exclusiva tras cada inserción exitosa. Al concluir, la función retorna el valor entero de `j`, determinando con precisión la nueva longitud lógica del arreglo filtrado.

4. Persistencia Segura y Formateo de Archivos de Texto

La función `guardar` administra la persistencia de datos en el almacenamiento secundario del equipo. Se declara un apuntador `FILE *f` y se invoca la apertura mediante `fopen("resultado.txt", "w")`. Con el fin de evitar desbordamientos e instrucciones inválidas, se anidó un bloque de validación condicional `if (f != NULL)`.

Dentro del ciclo de escritura, se corrigió el error crítico de aglutinamiento de datos modificando la instrucción de salida por la sentencia estructurada `fprintf(f, "%d ", v[i]);`. Al incrustar un espacio en blanco explícito posterior al especificador de formato numérico `%d`, se asegura que cada elemento se resguarde de manera independiente en el archivo plano. Finalmente, se invoca `fclose(f)` para liberar el búfer del sistema y se añade un bloque `else` de contingencia para notificar al usuario en pantalla si ocurriese un fallo imprevisto de permisos de disco.

```

#include <stdio.h>

#define TAMANO 6

void capturar(int v[], int n) {
    for(int i = 0; i < n; i++) {
        printf("Ingresa el numero %d: ", i + 1);
        scanf("%d", &v[i]);
    }
}

void mostrar(int v[], int n) {
    for(int i = 0; i < n; i++) {
        printf("%d ", v[i]);
    }
    printf("\n");
}

int filtrar(int origen[], int destino[], int n) {
    int j = 0;
    for(int i = 0; i < n; i++) {
        if(origen[i] % 2 != 0) {
            destino[j] = origen[i];
            j++;
        }
    }
    return j;
}

void guardar(int v[], int n) {
    FILE *f;
    f = fopen("resultado.txt", "w");

    if (f != NULL) {
        for(int i = 0; i < n; i++) {
            fprintf(f, "%d ", v[i]); // Espacio agregado para formateo legible
        }
        fclose(f);
    } else {
        printf("Error: No se pudo crear o abrir el archivo resultado.txt\n");
    }
}

int main() {
    int arregloOriginal[TAMANO];
    int arregloFiltrado[TAMANO];
    int totalFiltrados;

    capturar(arregloOriginal, TAMANO);

    printf("\nArreglo original:\n");
    mostrar(arregloOriginal, TAMANO);

    totalFiltrados = filtrar(arregloOriginal, arregloFiltrado, TAMANO);

    printf("\nArreglo filtrado (Impares):\n");
    mostrar(arregloFiltrado, totalFiltrados);

    guardar(arregloFiltrado, totalFiltrados);
}

```

```
printf("\nProceso terminado. Revisa el archivo resultado.txt\n");  
  
return 0;  
}
```

CONCLUSIÓN

El desarrollo de esta actividad de laboratorio consolidó de manera definitiva mis competencias en la estructuración de software robusto en lenguaje C. Migrar de una programación lineal y masiva dentro de `main` hacia un modelo basado en responsabilidades modulares aisladas me demostró las inmensas ventajas de la abstracción: el código se vuelve sumamente sencillo de depurar, probar y mantener.

Un aprendizaje crítico de esta sesión fue entender el impacto directo de la sintaxis sobre la persistencia en archivos de texto plano. Al examinar el comportamiento inicial de la función de guardado, comprendí que la computadora procesa de forma literal las instrucciones de flujo; omitir un delimitador provoca que múltiples registros numéricos se fusionen físicamente en el disco, arruinando la fidelidad de la información. Al corregir esto con un espacio adaptado en el formato de `fprintf` y blindar la operación con un filtro de validación para punteros nulos (`NULL`), logré construir una solución segura, escalable y con persistencia real. Estas técnicas constituyen la base de la ingeniería de software estructural, preparándome para abordar operaciones de lectura avanzada y manipulación de archivos binarios complejos.

REFERENCIAS

- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- Deitel, P., & Deitel, H. (2016). *C How to Program* (8th ed.). Pearson.